

자료구조 실습 보고서

[제 13 주] 리스트 반복자

제출일 : 2018 년 6 월 5 일

201702081 최재범

1. 프로그램 설명서

a. 주요 알고리즘 / 자료구조

Iterator 를 구현하는 두 멤버 함수는 리스트의 종류에 따라 다르게 동작한다.

- `public boolean hasNext()`

ArrayListIterator : 현재 인덱스를 가리키는 `_nextPosition` 필드가 리스트의 크기보다 작으면 `true` 를 반환한다.

LinkedListIterator : 현재 노드를 가리키는 `_next` 필드가 `null` 이 아니면 `true` 를 반환한다.

- `public T next()`

ArrayListIterator : 현재 인덱스를 가리키는 `_nextPosition` 필드가 리스트가 사이즈와 같다면 `null` 을 반환하고,

아니라면 현재 `_nextPosition` 에 존재하는 원소를 반환함과 동시에 `_nextPosition` 을 1 증가시킨다.

LinkedListIterator : 현재 노드를 가리키는 `_next` 필드가 `null` 이면 `null` 을 반환하고,

아니라면 현재 `_next` 가 가리키는 노드의 원소를 반환함과 동시에 `_next` 를 한 번 이동시킨다.

b. 함수 설명서

① ArrayListIterator

■ ApplicationController

- `public void run()` : 프로그램 실행

■ AppView

- `public int inputInt()` : 정수 입력받아 반환
- `public int inputNumber()` : 메시지 출력 후 정수 입력받아 반환
- `public String inputString()` : 문자열 입력받아 반환
- `public char inputCharacter()` : 문자 1 개 입력받아 반환
- `public void outputAdd(char anElement)` : 리스트에 추가된 원소 출력
- `public void outputElement(char anElement)` : 리스트의 원소 하나 출력
- `public void outputSize(int aStackSize)` : 리스트의 현재 크기 출력
- `public void outputRemove(char anElement)` : 제거된 원소 출력

- public void outputMessage(String aMessageString) : 문자열 출력

■ SortedArrayList

- public SortedArrayList() : 생성자 - 리스트의 최대 크기를 기본값으로 함
- public SortedArrayList(int givenMaxSize) : 생성자 - 리스트의 최대 크기를 사용자가 지정
- public int size() : 리스트의 현재 크기 반환
- public boolean contains(T anElement) : 주어진 원소의 리스트에서의 존재 여부 반환
- public boolean isFull() : 리스트가 다 찼는지 여부 반환
- public boolean isEmpty() : 리스트가 비어 있는지 여부 반환
- public boolean add(T anElement) : 오름차순에 맞도록 리스트에 원소 추가
- public T removeFrom(int aPosition) : 주어진 위치의 원소를 리스트에서 제거
- public T removeMin() : 리스트에서 최솟값 제거
- public T removeMax() : 리스트에서 최댓값 제거
- public void clear() : 리스트 초기화
- public T elementAt(int anOrder) : 주어진 위치의 원소 반환
- public ListIterator<T> listIterator() : ListIterator 의 생성자를 호출하여, 리스트 내 원소의 순차적 탐색을 위한 Iterator 객체의 반환

■ ListIterator (SortedArrayList 의 내부 클래스)

- public boolean hasNext() : 현재 위치가 존재하는지 여부 반환
- public T next() : 현재 위치의 원소를 반환하며 다음 위치로 이동

② LinkedListIterator

SortedArrayList -> SortedLinkedList 로의 변경, Node 클래스의 추가 외에는 위와 전부 동일하다.

■ SortedLinkedList

- public SortedLinkedList() : 생성자 - 리스트의 최대 크기를 기본값으로 함
- public SortedLinkedList(int givenMaxSize) : 생성자 - 리스트의 최대 크기를 사용자가 지정
- public int size() : 리스트의 현재 크기 반환
- public boolean contains(T anElement) : 주어진 원소의 리스트에서의 존재 여부 반환
- public boolean isFull() : 리스트가 다 찼는지 여부 반환
- public boolean isEmpty() : 리스트가 비어 있는지 여부 반환
- public boolean add(T anElement) : 오름차순에 맞도록 리스트에 원소 추가
- public T removeFrom(int aPosition) : 주어진 위치의 원소를 리스트에서 제거
- public T removeMin() : 리스트에서 최솟값 제거
- public T removeMax() : 리스트에서 최댓값 제거
- public void clear() : 리스트 초기화
- public T elementAt(int anOrder) : 주어진 위치의 원소 반환

■ Node

- `public Node()` : 디폴트 생성자
- `public Node(T givenElement)` : 노드에 담을 원소만 설정하고, 다음 노드의 위치 정보는 설정하지 않음 (생성자)
- `public Node(T givenElement, Node<T> givenNode)` : 노드에 담을 정보와 다음 노드의 위치 정보 설정 (생성자)
- `public T element()` : 노드에 담긴 원소 반환 (Getter)
- `public Node<T> next()` : 다음 노드의 위치 정보 반환 (Getter)
- `public void setElement(T newElement)` : 노드에 원소 설정 (Setter)
- `public void setNext(Node<T> newNext)` : 노드에 다음 노드의 위치 정보 설정 (Setter)

c. 종합 설명서

사용자의 관점에서 리스트의 원소에 접근하려면 내부 구조를 알아야 하는 경우가 있다. 하지만 이는 캡슐화의 측면에서 부적절하기 때문에 Iterator 를 사용하여 리스트의 원소에 순차적으로 접근하도록 하였다.

% : 정수 입력받아 추가
~ : 리스트 초기화
- : 리스트에서 최솟값 삭제
+ : 리스트에서 최댓값 삭제
: 리스트의 현재 크기 출력
? : 특정 위치의 원소 제거
/ : 처음부터 끝까지 전부 출력
! : 종료

2. 프로그램 장단점 분석

배열을 이용하면 특정 위치에 대한 접근이 빠르지만, 처음 또는 중간에 삽입 / 제거할 때 시간이 오래 걸리며 크기가 제한적이다.

링크드 리스트를 이용하면 크기에 대한 제한이 없고 삽입 / 제거할 때 시간이 적게 걸리지만, 특정 위치에 대한 접근이 느리다.

3. 실행 결과 분석

a. 입력과 출력

```

< 리스트를 시작합니다. >
> 문자를 입력하십시오 : %
- 숫자를 입력하십시오 : 5
[Insert] 삽입된 원소는 5입니다.

> 문자를 입력하십시오 : %
- 숫자를 입력하십시오 : 4
[Insert] 삽입된 원소는 4입니다.

> 문자를 입력하십시오 : %
- 숫자를 입력하십시오 : 3
[Insert] 삽입된 원소는 3입니다.

> 문자를 입력하십시오 : %
- 숫자를 입력하십시오 : 2
[Insert] 삽입된 원소는 2입니다.

> 문자를 입력하십시오 : %
- 숫자를 입력하십시오 : 1
[Insert] 삽입된 원소는 1입니다.

> 문자를 입력하십시오 : /
[LIST]      1      2      3      4      5

> 문자를 입력하십시오 : +
[Delete] 삭제된 원소는 5입니다.

> 문자를 입력하십시오 : -
[Delete] 삭제된 원소는 1입니다.

> 문자를 입력하십시오 : #
[Length] 리스트에는 현재 3 개가 존재합니다.

> 문자를 입력하십시오 : /
[LIST]      2      3      4

> 문자를 입력하십시오 : ?
- 숫자를 입력하십시오 : 1
[Delete] 삭제된 원소는 3입니다.

> 문자를 입력하십시오 : ~
- 리스트를 초기화 합니다.

> 문자를 입력하십시오 : /
[LIST]

> 문자를 입력하십시오 : !
< 리스트를 종료합니다. >

```

두 프로그램의 결과는 같다.

b. 결과분석

5, 4, 3, 2, 1 을 차례대로 삽입하고 확인해보면 1, 2, 3, 4, 5 로 정렬이 되었고,
최댓값과 최솟값을 삭제하면 각각 5, 1 이 삭제되고 2, 3, 4 로 3 개가 남는다.

order 1 의 값을 삭제하면 두 번째 값인 3 이 삭제되고, 초기화도 정상적으로 된 것을 알 수 있다.
따라서 결과는 유효하다.

4. 소스코드

■ ArrayListIterator

AppController.java

```

public class AppController {

    private AppView _appView;
    private List<Integer> _sortedList;

    public AppController() {
        this._appView = new AppView();
    }

    public void run() {

        this._sortedList = new SortedArrayList<Integer>();
        char command = 0;
        int input;
    }
}

```

```

    this.showMessage(MessageID.Notice_StartProgram);
    while(command != '!') {

        command = this._appView.inputCharacter();

        if(command == '%') {
            input = this._appView.inputNumber();
            this.add(input);
        }

        else if(command == '~')
            this.reset();

        else if(command == '-')
            this.removeMin();

        else if(command == '+')
            this.removeMax();

        else if(command == '#')
            this.showSize();

        else if(command == '?') {
            input = this._appView.inputNumber();
            this.removeFrom(input);
        }

        else if(command == '/')
            this.showAll();

        else if(command == '!')
            break;

        else
            this.showMessage(MessageID.Error_WrongMenu);

    }

    this.showMessage(MessageID.Notice_EndProgram);

}

// 비공개 함수
private void showSize() {
    this._appView.outputSize(this._sortedList.size());
}

private void reset() {
    this.showMessage(MessageID.Notice_Reset);
    this._sortedList.clear();
}

private void showAll() {
    this.showMessage(MessageID.Notice_ShowStartList);

    /*
    for(int i = 0; i < this._sortedList.size(); i++) {
        this._appView.outputElement(this._sortedList.elementAt(i));
    }
    */
}

```

```

        Iterator<Integer> iterator = this._sortedList.listIterator();
        while(iterator.hasNext()) {
            this._appView.outputElement( (int)iterator.next() );
        }

        this.showMessage(MessageID.Notice_ShowEndList);
    }

    private void add(int inputValue) {
        Integer inputElement = inputValue;
        boolean isAdded = this._sortedList.add(inputElement);

        if(!isAdded) {
            this.showMessage(MessageID.Error_InputFull);
        }
        else {
            this._appView.outputAdd(inputElement);
        }
    }

    private void removeMin() {
        if(this._sortedList.isEmpty()) {
            this.showMessage(MessageID.Error_Empty);
        }
        else {
            int removedMin = this._sortedList.removeMin();
            this._appView.outputRemove(removedMin);
        }
    }

    private void removeMax() {
        if(this._sortedList.isEmpty()) {
            this.showMessage(MessageID.Error_Empty);
        }
        else {
            int removedMax = this._sortedList.removeMax();
            this._appView.outputRemove(removedMax);
        }
    }

    private void removeFrom(int aPosition) {
        if(this._sortedList.isEmpty()) {
            this.showMessage(MessageID.Error_Empty);
        }
        else {
            int removedElement = this._sortedList.removeFrom(aPosition);
            this._appView.outputRemove(removedElement);
        }
    }

    private void showMessage(MessageID aMessageID) {
        switch(aMessageID) {

            case Notice_StartProgram:
                this._appView.outputMessage("< 리스트를 시작합니다. > \n");
                break;

            case Notice_EndProgram:

```

```

        this._appView.outputMessage("\n< 리스트를 종료합니다. > \n");
        break;

    case Notice_Reset:
        this._appView.outputMessage("- 리스트를 초기화 합니다. \n");
        break;

    case Notice_ShowStartList:
        this._appView.outputMessage("[LIST] ");
        break;

    case Notice_ShowEndList:
        this._appView.outputMessage("\n");
        break;

    case Error_WrongMenu:
        this._appView.outputMessage("! 잘못된 메뉴를 입력하였습니다. \n");
        break;

    case Error_InputFull:
        this._appView.outputMessage("! 리스트가 꽉 찼습니다. \n");
        break;

    case Error_Empty:
        this._appView.outputMessage("! 리스트가 비어 있습니다. \n");
        break;

    }

}

}

```

AppView.java

```

import java.util.Scanner;

public class AppView {

    private Scanner _scanner;

    public AppView() {
        this._scanner = new Scanner(System.in);
    }

    // 입력 관련

    public String inputString() {
        return this._scanner.nextLine();
    }

    public int inputInt() {
        return Integer.parseInt(this._scanner.nextLine());
    }

    public char inputCharacter() {
        System.out.print("\n> 문자를 입력하시오 : ");
    }
}

```



```

        return this._scanner.nextLine().charAt(0);
    }

    public int inputNumber() {
        System.out.print("- 숫자를 입력하시오 : ");
        return Integer.parseInt(this._scanner.nextLine());
    }

    // 출력 관련

    public void outputSize(int size) {
        System.out.println("[Length] 리스트에는 현재 " + size + " 개가 존재합니다.");
    }

    public void outputAdd(int anElement) {
        System.out.println("[Insert] 삽입된 원소는 " + anElement + "입니다.");
    }

    public void outputRemove(int anElement) {
        System.out.println("[Delete] 삭제된 원소는 " + anElement + "입니다.");
    }

    public void outputElement(int anElement) {
        System.out.printf("%3d ", anElement);
    }

    public void outputMessage(String aMessage) {
        System.out.print(aMessage);
    }
}

```

List.java

```

public interface List<T> {

    // % : 정수 입력 관련
    public boolean add(T anElement);
    public boolean contains(T anElement);
    public boolean isFull();

    // ~ : 초기화 관련
    public void clear();

    // - : 최솟값 삭제 관련
    public T removeMin();
    public boolean isEmpty();

    // + : 최댓값 삭제 관련
    public T removeMax();
}

```

```

/*      public boolean isEmpty(); */

// # : 리스트 길이 출력 관련
public int size();

// ? : 특정 순서의 값 삭제 관련
public T removeFrom(int aPosition);

// / : 리스트의 원소 전부 출력 관련

/* public T elementAt(int anOrder); */

public Iterator<T> listIterator();
}

```

SortedList.java

```

public class SortedArrayList<T extends Comparable<T>> implements List<T> {

    private static final int DEFAULT_MAX_SIZE = 20;
    private int _maxSize;
    private int _size;
    private T[] _element;

    // 생성자
    @SuppressWarnings("unchecked")
    public SortedArrayList() {
        this._maxSize = DEFAULT_MAX_SIZE;
        this._size = 0;
        this._element = (T[])new Comparable[this._maxSize];
    }

    @SuppressWarnings("unchecked")
    public SortedArrayList(int givenMaxSize) {
        this._maxSize = givenMaxSize;
        this._size = 0;
        this._element = (T[])new Comparable[this._maxSize];
    }

    @Override
    public int size() {
        return this._size;
    }

    @Override
    public boolean contains(T anElement) {
        boolean found = false;
        for(int index = 0; index < this._size; index++) {
            if(this._element[index].compareTo(anElement) == 0) {
                found = true;
            }
        }
    }
}

```

```

    }
    return found;
}

@Override
public boolean isFull() {
    return this._size == this._maxSize;
}

@Override
public boolean isEmpty() {
    return this._size == 0;
}

@Override
public boolean add(T anElement) {

    if(this.isFull()) {
        return false;
    }

    if(this.isEmpty()) {
        this._element[0] = anElement;
        this._size++;
        return true;
    }

    else {
        int currentSize = this._size;

        // 탐색
        for(int i = 0; i < currentSize; i++) {

            // i 번째에 더 큰 수가 있으면, 공간을 만들고 그 이전에 삽입
            if(this._element[i].compareTo(anElement) >= 0) {

                // 끝부터 한 칸씩 당김
                for(int shiftIndex = this._size - 1; shiftIndex >= i; shiftIndex--) {
                    this._element[shiftIndex + 1] = this._element[shiftIndex];
                }

                this._element[i] = anElement;
                this._size++;
                return true;
            }
        }

        // 리스트에서 더 큰 수를 찾지 못하면, 맨 끝에다가 삽입
        this._element[this._size] = anElement;
        this._size++;
        return true;
    }
}

@Override
public T removeFrom(int aPosition) {
    if(this.isEmpty()) {

```

```

        return null;
    }
    else {
        if(aPosition < 0 || aPosition >= this._size) {
            return null;
        }
        else {
            T removedElement = this._element[aPosition];
            for(int shiftIndex = aPosition; shiftIndex < this._size - 1; shiftIndex++) {
                this._element[shiftIndex] = this._element[shiftIndex + 1];
            }
            this._element[this._size - 1] = null;
            this._size--;
            return removedElement;
        }
    }
}

```

```

@Override
public T removeMin() {
    int minPosition = 0;
    for(int i = 1; i < this._size; i++) {
        if( this._element[i].compareTo(this._element[minPosition]) < 0 )
            minPosition = i;
    }

    T removedElement = this.removeFrom(minPosition);
    return removedElement;
}

```

```

@Override
public T removeMax() {
    int maxPosition = 0;
    for(int i = 1; i < this._size; i++) {
        if( this._element[i].compareTo(this._element[maxPosition]) > 0 )
            maxPosition = i;
    }

    T removedElement = this.removeFrom(maxPosition);
    return removedElement;
}

```

```

@Override
public void clear() {
    this._size = 0;
    for(int i = 0; i < this._maxSize; i++) {
        this._element[i] = null;
    }
}

```

```

public T elementAt(int anOrder) {
    if(anOrder < 0 || anOrder >= this._size) {
        return null;
    }
    else {
        return this._element[anOrder];
    }
}

```

```

// Iterator

public ListIterator<T> listIterator() {
    return new ListIterator();
}

public class ListIterator<T> implements Iterator<T> {

    private int _nextPosition;

    private ListIterator() {
        this._nextPosition = 0;
    }

    public boolean hasNext() {
        return ( this._nextPosition < SortedArrayList.this.size() );
    }

    // next에 들은 값 반환하면서 next 이동
    @SuppressWarnings("unchecked")
    public T next() {
        if(this._nextPosition == SortedArrayList.this.size()) {
            return null;
        }
        else {
            T element = (T)SortedArrayList.this._element[this._nextPosition];
            this._nextPosition++;
            return element;
        }
    }
}
}

```

Iterator.java

```

public interface Iterator<T> {

    public boolean hasNext();
    public T next();
}

```

MessageID.java

```

public enum MessageID {

    // Message IDs for Notices
    Notice_StartProgram,
    Notice_EndProgram,
    Notice_Reset,
    Notice_ShowStartList,
    Notice_ShowEndList,
}

```

```

// Message IDs for Errors
Error_WrongMenu,
Error_InputFull,
Error_Empty

}

```

DS1_13_201702081_최재범.java

```

public class DS1_13_201702081_최재범 {

    public static void main(String[] args) {

        AppController appController = new AppController();
        appController.run();

    }

}

```

■ LinkedListIterator

AppController.java

```

public class AppController {

    private AppView _appView;
    private List<Integer> _sortedList;

    public AppController() {
        this._appView = new AppView();
    }

    public void run() {

        this._sortedList = new SortedLinkedList<Integer>();
        char command = 0;
        int input;

        this.showMessage(MessageID.Notice_StartProgram);
        while(command != '!') {

            command = this._appView.inputCharacter();

            if(command == '%') {
                input = this._appView.inputNumber();
                this.add(input);
            }

            else if(command == '~')
                this.reset();

            else if(command == '-')
                this.removeMin();

            else if(command == '+')
                this.removeMax();

            else if(command == '#')
                this.showSize();

            else if(command == '?') {

```

```

        input = this._appView.inputNumber();
        this.removeFrom(input);
    }

    else if(command == '/')
        this.showAll();

    else if(command == '!')
        break;

    else
        this.showMessage(MessageID.Error_WrongMenu);

    }

    this.showMessage(MessageID.Notice_EndProgram);

}

// 비공개 함수
private void showSize() {
    this._appView.outputSize(this._sortedList.size());
}

private void reset() {
    this.showMessage(MessageID.Notice_Reset);
    this._sortedList.clear();
}

private void showAll() {
    this.showMessage(MessageID.Notice_ShowStartList);

    /*
    for(int i = 0; i < this._sortedList.size(); i++) {
        this._appView.outputElement(this._sortedList.elementAt(i));
    }
    */

    Iterator<Integer> iterator = this._sortedList.listIterator();
    while(iterator.hasNext()) {
        this._appView.outputElement( (int)iterator.next() );
    }

    this.showMessage(MessageID.Notice_ShowEndList);
}

private void add(int inputValue) {
    Integer inputElement = inputValue;
    boolean isAdded = this._sortedList.add(inputElement);

    if(!isAdded) {
        this.showMessage(MessageID.Error_InputFull);
    }
    else {
        this._appView.outputAdd(inputElement);
    }
}
}

```

```

private void removeMin() {
    if(this._sortedList.isEmpty()) {
        this.showMessage(MessageID.Error_Empty);
    }
    else {
        int removedMin = this._sortedList.removeMin();
        this._appView.outputRemove(removedMin);
    }
}

private void removeMax() {
    if(this._sortedList.isEmpty()) {
        this.showMessage(MessageID.Error_Empty);
    }
    else {
        int removedMax = this._sortedList.removeMax();
        this._appView.outputRemove(removedMax);
    }
}

private void removeFrom(int aPosition) {
    if(this._sortedList.isEmpty()) {
        this.showMessage(MessageID.Error_Empty);
    }
    else {
        int removedElement = this._sortedList.removeFrom(aPosition);
        this._appView.outputRemove(removedElement);
    }
}

private void showMessage(MessageID aMessageID) {
    switch(aMessageID) {

        case Notice_StartProgram:
            this._appView.outputMessage("< 리스트를 시작합니다. > \n");
            break;

        case Notice_EndProgram:
            this._appView.outputMessage("\n< 리스트를 종료합니다. > \n");
            break;

        case Notice_Reset:
            this._appView.outputMessage("- 리스트를 초기화 합니다. \n");
            break;

        case Notice_ShowStartList:
            this._appView.outputMessage("[LIST] ");
            break;

        case Notice_ShowEndList:
            this._appView.outputMessage("\n");
            break;

        case Error_WrongMenu:
            this._appView.outputMessage("! 잘못된 메뉴를 입력하였습니다. \n");
            break;

        case Error_InputFull:
            this._appView.outputMessage("! 리스트가 꽉 찼습니다. \n");

```



```

        break;

        case Error_Empty:
            this._appView.outputMessage("! 리스트가 비어 있습니다. \n");
            break;

    }

}

}

```

AppView.java

```

import java.util.Scanner;

public class AppView {

    private Scanner _scanner;

    public AppView() {
        this._scanner = new Scanner(System.in);
    }

    // 입력 관련

    public String inputString() {
        return this._scanner.nextLine();
    }

    public int inputInt() {
        return Integer.parseInt(this._scanner.nextLine());
    }

    public char inputCharacter() {
        System.out.print("\n> 문자를 입력하시오 : ");
        return this._scanner.nextLine().charAt(0);
    }

    public int inputNumber() {
        System.out.print("- 숫자를 입력하시오 : ");
        return Integer.parseInt(this._scanner.nextLine());
    }

    // 출력 관련

    public void outputSize(int size) {
        System.out.println("[Length] 리스트에는 현재 " + size + " 개가 존재합니다.");
    }

    public void outputAdd(int anElement) {
        System.out.println("[Insert] 삽입된 원소는 " + anElement + "입니다.");
    }
}

```

```

    public void outputRemove(int anElement) {
        System.out.println("[Delete] 삭제된 원소는 " + anElement + "입니다.");
    }

    public void outputElement(int anElement) {
        System.out.printf("%3d ", anElement);
    }

    public void outputMessage(String aMessage) {
        System.out.print(aMessage);
    }
}

```

List.java

```

public interface List<T> {

    // % : 정수 입력 관련
    public boolean add(T anElement);
    public boolean contains(T anElement);
    public boolean isFull();

    // ~ : 초기화 관련
    public void clear();

    // - : 최솟값 삭제 관련
    public T removeMin();
    public boolean isEmpty();

    // + : 최댓값 삭제 관련
    public T removeMax();
    public boolean isEmpty(); /*

    // # : 리스트 길이 출력 관련
    public int size();

    // ? : 특정 순서의 값 삭제 관련
    public T removeFrom(int aPosition);

    /// : 리스트의 원소 전부 출력 관련
    /* public T elementAt(int anOrder); */

    public Iterator<T> listIterator();
}

```

SortedLinkedList.java

```
public class SortedLinkedList<T> extends Comparable<T>> implements List<T> {

    private int _size;
    private Node<T> _head;

    // 생성자
    public SortedLinkedList() {
        this._size = 0;
        this._head = null;
    }

    @Override
    public int size() {
        return this._size;
    }

    @Override
    public boolean contains(T anElement) {
        boolean found = false;
        Node<T> nodeForSearch = this._head;

        while(nodeForSearch != null) {
            if(nodeForSearch.element().compareTo(anElement) == 0)
                found = true;
            nodeForSearch = nodeForSearch.next();
        }

        return found;
    }

    @Override
    public boolean isFull() {
        return false;
    }

    @Override
    public boolean isEmpty() {
        return this._size == 0;
    }

    @Override
    public boolean add(T anElement) {

        // 비어있을 때
        if(this.isEmpty()) {
            Node<T> newNode = new Node<T>(anElement);
            this._head = newNode;

            this._size++;
            return true;
        }
    }
}
```

```

// 1개 이상 있을 때
else {
    Node<T> newNode = new Node<T>(anElement);

    // 위치 찾기 : 이전 노드를 찾아야 삽입이 가능
    Node<T> previousPosition = new Node<T>(null, this._head);

    while( previousPosition.next() != null &&
           anElement.compareTo(previousPosition.next().element()) >= 0 )
    {
        previousPosition = previousPosition.next();
    }

    // 삽입 위치가 맨 앞
    if(previousPosition.next() == this._head) {
        newNode.setNext(this._head);
        this._head = newNode;
    }

    // 삽입 위치가 맨 끝
    else if(previousPosition.next() == null) {
        previousPosition.setNext(newNode);
    }

    // 삽입 위치가 중간
    else {
        newNode.setNext(previousPosition.next());
        previousPosition.setNext(newNode);
    }

    this._size++;
    return true;
}
}

```

```

@Override
public T removeFrom(int aPosition) {

    // 비어있을 때
    if(this.isEmpty()) {
        return null;
    }

    // 1개 이상 들어있을 때
    else {

        // 유효하지 않은 범위
        if(aPosition < 0 || aPosition >= this._size) {
            return null;
        }

        // 유효 범위
        else {

            T removedElement;

            // 맨 앞 원소 제거
            if(aPosition == 0) {
                removedElement = this._head.element();
            }

```

```

        this._head = this._head.next();
    }

    // 맨 앞이 아닌 원소 제거
    else {

        // 직전까지 이동
        Node<T> previousNode = this._head;
        for(int i = 0; i < aPosition - 1; i++)
            previousNode = previousNode.next();

        // 맨 끝의 원소일 경우
        if(aPosition == this._size - 1) {
            removedElement = previousNode.next().element();
            previousNode.setNext(null);
        }

        // 맨 끝이 아닌 원소일 경우
        else {
            removedElement = previousNode.next().element();
            previousNode.setNext(previousNode.next().next());
        }

    }

    this._size--;
    return removedElement;
}

}

@Override
public T removeMin() {
    int minPosition = 0;
    for(int i = 1; i < this._size; i++) {
        if( this.elementAt(i).compareTo(this.elementAt(minPosition)) < 0 )
            minPosition = i;
    }

    T removedElement = this.removeFrom(minPosition);
    return removedElement;
}

@Override
public T removeMax() {
    int maxPosition = 0;
    for(int i = 1; i < this._size; i++) {
        if( this.elementAt(i).compareTo(this.elementAt(maxPosition)) > 0 )
            maxPosition = i;
    }

    T removedElement = this.removeFrom(maxPosition);
    return removedElement;
}

@Override
public void clear() {
    this._size = 0;
    this._head = null;
}

```

```

    }

    public T elementAt(int anOrder) {
        if(anOrder < 0 || anOrder >= this._size) {
            return null;
        }
        else {
            Node<T> searchNode = this._head;
            for(int i = 1; i <= anOrder; i++)
                searchNode = searchNode.next();

            return searchNode.element();
        }
    }
}

// Iterator

public ListIterator<T> listIterator() {
    return new ListIterator();
}

public class ListIterator<T> implements Iterator<T> {

    private Node<T> _next;

    @SuppressWarnings("unchecked")
    private ListIterator() {
        this._next = (Node<T>) SortedLinkedList.this._head;
    }

    public boolean hasNext() {
        return ( this._next != null );
    }

    // next에 들은 값 반환하면서 next 이동
    public T next() {
        if(this._next == null) {
            return null;
        }
        else {
            T element = this._next.element();
            this._next = this._next.next();
            return element;
        }
    }
}
}

```

Node.java

```

public class Node<T>
{

```

```

private T _element;
private Node<T> _next;

// Constructor
public Node() {}

public Node(T givenElement) {
    this._element = givenElement;
    this._next = null;
}

public Node(T givenElement, Node<T> givenNext) {
    this._element = givenElement;
    this._next = givenNext;
}

// Getter
public T element() {
    return this._element;
}
public Node<T> next() {
    return this._next;
}

// Setter
public void setElement(T newElement) {
    this._element = newElement;
}
public void setNext(Node<T> newNext) {
    this._next = newNext;
}
}

```

Iterator.java

```

public interface Iterator<T> {

    public boolean hasNext();
    public T next();

}

```

MessageID.java

```

public enum MessageID {

    // Message IDs for Notices
    Notice_StartProgram,
    Notice_EndProgram,
    Notice_Reset,
    Notice_ShowStartList,
    Notice_ShowEndList,

    // Message IDs for Errors
    Error_WrongMenu,
    Error_InputFull,
    Error_Empty
}

```

```
}
```

DS1_13_201702081_최재범.java

```
public class DS1_13_201702081_최재범 {  
  
    public static void main(String[] args) {  
  
        AppController appController = new AppController();  
        appController.run();  
  
    }  
  
}
```